SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	N.Ganitha	**Reg. No.:**	192425235
Section / Batch:	2024	**Date:**	28-10-26

Code of Conduct

*I **N.Ganitha(192425235)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: N.Ganitha

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:
Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
Password
Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)
Mobile Number
Must contain exactly 10 digits.
First digit should be between 6–9.
Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. **Problem Statement**

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
    - States
    - Input alphabet
    - Transition table
    - Initial state

    - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input
abaab

Output
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. **Problem Statement**

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output
Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Section 1: Intelligent Email and Password Validator Using Regular Expressions

Aim

To develop a Python application using regular expressions to validate email addresses, passwords, and mobile numbers according to the given security rules.

Algorithm

1. Read the email address, password, and mobile number.
2. Use a regular expression to check whether the email follows the specified format.
3. Check that the password has at least eight characters, including uppercase, lowercase, digit, and allowed special character.
4. Check that the mobile number contains exactly 10 digits and starts with a digit from 6 to 9.
5. Display the validation results.

Python Program

```
import re
email = input("Enter email: ")
password = input("Enter password: ")
mobile = input("Enter mobile number: ")
email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!])[A-Za-z\d@#$%&!]{8,}$'
mobile_pattern = r'^[6-9]\d{9}$'
if re.fullmatch(email_pattern, email, re.IGNORECASE):
    print("Valid Email")
else:
    print("Invalid Email")
if re.fullmatch(password_pattern, password):
    print("Strong Password")
```

```
else:
    print("Weak Password")
if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Sample Input

Enter email: ganitha123@gmail.com

Enter password: Ganitha@123

Enter mobile number: 9876543210

Expected Output

Valid Email

Strong Password

Valid Mobile Number

Test Cases

- ganitha@gmail.com → Valid Email
- 123ganitha@gmail.com → Invalid Email
- Ganitha@123 → Strong Password
- ganitha123 → Weak Password
- 9876543210 → Valid Mobile Number
- 1234567890 → Invalid Mobile Number

How this meets Level 4

- Uses regular expressions for all three validations.
- Checks every specified security rule.

- Uses full-string matching to reject extra or invalid characters.

Section 2: Design a Finite State Automata (FSA) Simulator

Aim

To develop a Python-based Deterministic Finite Automaton (DFA) simulator that accepts a DFA definition, processes multiple input strings, displays state transitions, and determines whether each string is accepted or rejected.

Algorithm

1. Read the states and input alphabet.
2. Read the transition table, initial state, and final states.
3. Accept multiple input strings.
4. Start from the initial state and process each input symbol.
5. Follow the transition table and display the transition path.
6. If a transition is missing, reject the string.
7. After processing all symbols, accept the string if the current state is a final state; otherwise, reject it.

Python Program

```
states = input("Enter states: ").split()
alphabet = input("Enter input alphabet: ").split()
initial = input("Enter initial state: ")
finals = set(input("Enter final states: ").split())
n = int(input("Enter number of transitions: "))
transitions = {}
print("Enter each transition as: current_state symbol next_state")
for _ in range(n):
```

```
    source, symbol, destination = input().split()
    transitions[(source, symbol)] = destination
count = int(input("Enter number of input strings: "))
for _ in range(count):
    string = input("Enter string (use # for empty string): ")
    if string == "#":
        string = ""
    current = initial
    path = [current]
    valid = True
    for symbol in string:
        if symbol not in alphabet:
            valid = False
            break
        key = (current, symbol)
        if key not in transitions:
            valid = False
            break
        current = transitions[key]
        path.append(current)
    print("Transition Path:", " -> ".join(path))
    if valid and current in finals:
        print("Accepted")
    else:
        print("Rejected")
```

Sample Input

This DFA accepts strings ending with ab, using alphabet {a, b}.

Enter states: q0 q1 q2
Enter input alphabet: a b

Enter initial state: q0

Enter final states: q2

Enter number of transitions: 6

Enter each transition as: current_state symbol next_state

q0 a q1

q0 b q0

q1 a q1

q1 b q2

q2 a q1

q2 b q0

Enter number of input strings: 3

Enter string (use # for empty string): abab

Enter string (use # for empty string): abaab

Enter string (use # for empty string): aabb

Expected Output

Transition Path: q0 -> q1 -> q2 -> q1 -> q2

Accepted

Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2

Accepted

Transition Path: q0 -> q1 -> q1 -> q2 -> q0

Rejected

Test Cases

- ab → Accepted
- aab → Accepted
- abab → Accepted
- aabb → Rejected
- Empty string (#) → Rejected

- A string containing a symbol outside the alphabet → Rejected

How this meets Level 4

- Supports a user-defined DFA rather than a hard-coded example.
- Processes multiple strings in one execution.
- Displays the complete transition path.
- Correctly handles empty strings, invalid symbols, and missing transitions.
- Uses a transition dictionary for efficient lookup.

Section 3: Smart Pattern Matching Engine Using Regular Expressions

Aim

To develop a Python-based text search engine using regular expressions to identify words, prefixes, suffixes, dates, phone numbers, hashtags, and mentions from a given text.

Algorithm

1. Read the input text.
2. Display a menu of search operations.
3. Accept the user's choice.
4. Apply the corresponding regular expression to find matching patterns.
5. For prefix and suffix searches, accept a search term and match words beginning or ending with it.
6. Display all matches or a message if no matches are found.

Python Program

```
import re
text = input("Enter text: ")
print("\n1. Word Search")
print("2. Search Date")
print("3. Search Phone Number")
print("4. Search Hashtag")
print("5. Search Mention")
print("6. Search Prefix")
print("7. Search Suffix")
choice = input("Enter your choice: ")
patterns = {
    "1": r'\b[A-Za-z]+\b',
    "2": r'\b\d{2}/\d{2}/\d{4}\b',
    "3": r'\b[6-9]\d{9}\b',
    "4": r'(?<!\w)#[A-Za-z0-9_]+',
    "5": r'(?<!\w)@[A-Za-z0-9_]+'
}
if choice in patterns:
    matches = re.findall(patterns[choice], text)

elif choice in ("6", "7"):
    term = input("Enter prefix/suffix: ")
    if not term or not re.fullmatch(r'[A-Za-z0-9_]+', term):
        matches = []
    else:
        words = re.findall(r'\b[A-Za-z0-9_]+\b', text)
        if choice == "6":
            matches = [w for w in words
                       if w.lower().startswith(term.lower())]
        else:
```

```
        matches = [w for w in words
                   if w.lower().endswith(term.lower())]
else:
    matches = None
if matches is None:
    print("Invalid Choice")
elif matches:
    print("Matching Patterns:", matches)
else:
    print("No Matches Found")
```

Sample Input

Enter text: Meeting on 12/09/2026. Call 9876543210.

#NLP @OpenAI natural language processing

Enter your choice: 2

Expected Output

Matching Patterns: ['12/09/2026']

Additional Test Cases

- Choice 3 → ['9876543210']
- Choice 4 → ['#NLP']
- Choice 5 → ['@OpenAI']
- Choice 6, prefix nat → ['natural']
- Choice 7, suffix ing → ['Meeting', 'processing']
- Choice 1 → Extracts all alphabetic words.
- Invalid menu choice → Invalid Choice
- No matching pattern → No Matches Found

How this meets Level 4

- Supports all seven search operations.
- Uses regular expressions for efficient pattern extraction.
- Handles case-insensitive prefix and suffix matching.
- Supports repeated matches and reports when no matches are found.
- Includes invalid-choice handling and tests different search conditions.